

第7章

搜索树

从本章开始，讨论的重点将逐步转入查找技术。实际上，此前的若干章节已经就此做过一些讨论，在向量与列表等结构中，甚至已经提供并实现了对应的ADT接口。然而遗憾的是，此前这类接口的总体效率均无法令人满意。

以31页代码2.1中的向量模板类Vector为例，其中针对无序和有序向量的查找，分别提供了find()和search()接口。前者的实现策略只不过是目标对象与向量内存放的对象逐个比对，故最坏情况下需要运行 $O(n)$ 时间。后者利用二分查找策略尽管可以确保在 $O(\log n)$ 时间内完成单次查找，但一旦向量本身需要修改，无论是插入还是删除，在最坏情况下每次仍需 $O(n)$ 时间。而就代码3.2中的列表模板类List(70页)而言，情况反而更糟：即便不考虑对列表本身的修改，无论find()或search()接口，在最坏情况或平均情况下都需要线性的时间。另外，基于向量或列表实现的栈和队列，一般地甚至不提供对任意成员的查找接口。总之，若既要求对象集合的组成可以高效率地动态调整，同时也要求能够高效率地查找，则以上线性结构均难以胜任。

那么，高效率的动态修改和高效率的静态查找，究竟能否同时兼顾？如有可能，又应该采用什么样的数据结构？接下来的两章，将逐步回答这两个层次的问题。

因为这部分内容所涉及的数据结构变种较多，它们各具特色、各有所长，也有其各自的适用范围，故按基本和高级两章分别讲解，相关内容之间的联系如图7.1所示。

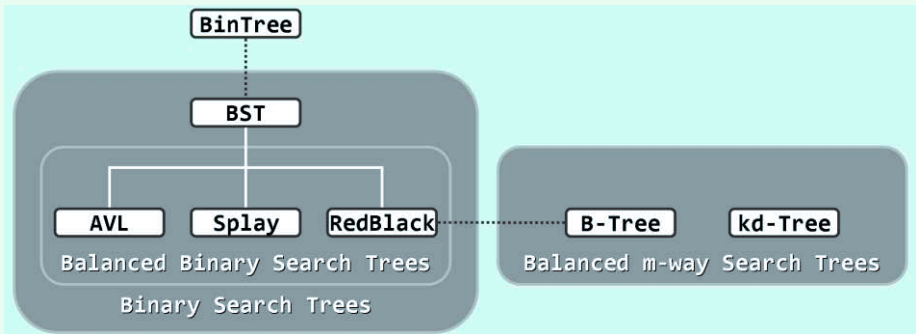


图7.1 第7章和第8章内容纵览

本章将首先介绍树式查找的总体构思、基本算法以及数据结构，通过对二分查找策略的抽象与推广，定义并实现二叉搜索树结构。尽管就最坏情况下的渐进时间复杂度而言，这一方法与此前相比并无实质的改进，但这部分内容依然十分重要——基于半线性的树形结构的这一总体构思，正是后续内容的立足点和出发点。比如，本章的后半部分将据此提出理想平衡和适度平衡等概念，并相应地引入和实现AVL树这一典型的平衡二叉搜索树。借助精巧的平衡调整算法，AVL树可以保证，即便是在最坏情况下，单次动态修改和静态查找也均可以在 $O(\log n)$ 时间内完成。这样，以上关于兼顾动态修改与静态查找操作效率的问题，就从正面得到了较为圆满的回答。接下来的第8章将在此基础上，针对更为具体的应用需求和更为精细的性能指标，介绍平衡搜索树家族的其它典型成员。